

Assignment 2: Deep Learning

Your Name

December 12, 2025

1 Feed-Forward Neural Network on MNIST

1.1 Program description and data preprocessing

The notebook `feedForwardMNIST.ipynb` implements a complete pipeline for classifying handwritten digits from the MNIST dataset using a feed-forward neural network.

The dataset is loaded using `torchvision.datasets.MNIST`. A transform function is applied to each image using `transforms.ToTensor()`, which converts the input from a PIL image with integer pixel values in the range $[0, 255]$ to a floating-point tensor with values scaled to $[0, 1]$. This normalization is necessary for stable gradient-based optimization and compatibility with PyTorch models.

Each image of size 28×28 is flattened into a 784-dimensional vector before being passed to the network.

1.2 Model architecture and parameter count

The feed-forward neural network consists of three fully connected layers:

- Linear layer mapping $784 \rightarrow 128$, followed by ReLU
- Linear layer mapping $128 \rightarrow 64$, followed by ReLU
- Linear layer mapping $64 \rightarrow 10$, followed by LogSoftmax

The number of parameters is computed as follows:

$$\text{Layer 1: } 784 \times 128 + 128 = 100,480$$

$$\text{Layer 2: } 128 \times 64 + 64 = 8,256$$

$$\text{Layer 3: } 64 \times 10 + 10 = 650$$

giving a total of **109,386 trainable parameters**.

1.3 Loss function and classification performance

The network outputs log-probabilities using `LogSoftmax`. Consequently, the loss function used is **Negative Log-Likelihood Loss (NLLLoss)**, which is appropriate when the model outputs log-probabilities.

The trained model achieves an overall classification accuracy of approximately 97–98% on the MNIST test set. Per-class accuracies are generally high, with digits such as 0 and 1 achieving the highest accuracy, while digits such as 5 and 8 are slightly more challenging due to their visual similarity to other digits.

1.4 Logits versus log-softmax outputs

If the final `LogSoftmax` layer is removed, the network outputs unnormalized scores known as *logits*. In this case, `NLLoss` is no longer appropriate, and the correct loss function to use is `CrossEntropyLoss`.

`CrossEntropyLoss` internally combines a log-softmax operation with the negative log-likelihood loss in a numerically stable manner. Working with logits is generally preferred over explicit softmax outputs because it improves numerical stability, avoids overflow issues, and leads to cleaner model definitions.

2 Convolutional Neural Network

2.1 Architecture and motivation

The convolutional neural network replaces the feed-forward network with the architecture specified in the assignment. It consists of two convolutional layers with ReLU activations and max-pooling, followed by a fully connected output layer.

The `Flatten` layer is required because convolutional layers output a four-dimensional tensor of shape (batch, channels, height, width), while fully connected layers expect a two-dimensional input of shape (batch, features).

2.2 Why the linear layer has 800 input features

Starting from an input image of size 28×28 :

- After the first convolution (kernel 3×3) and max-pooling, the feature map size becomes $16 \times 13 \times 13$.
- After the second convolution and max-pooling, the feature map size becomes $32 \times 5 \times 5$.

Flattening this output yields $32 \times 5 \times 5 = 800$ features, which explains the input size of the final linear layer.

2.3 Parameter count and performance comparison

The CNN has significantly fewer parameters than the feed-forward network:

- Conv layer 1: 160 parameters
- Conv layer 2: 4,640 parameters
- Linear layer: 8,010 parameters

for a total of **12,810 parameters**.

Despite having far fewer parameters, the CNN achieves a higher classification accuracy of approximately 99%. This improvement is due to the CNN's ability to exploit spatial locality and translation invariance in image data.

3 Autoencoders

3.1 Program structure and activations

The autoencoder consists of an encoder and a decoder, each implemented as a feed-forward network. The encoder uses linear layers with ReLU activations to project the input into a low-dimensional latent space. The decoder mirrors this structure and uses a Sigmoid activation in the output layer.

The Sigmoid activation is chosen because the input images are scaled to $[0, 1]$, ensuring that reconstructed pixel values lie in the same range.

3.2 Loss function and likelihood model

The loss function used is Binary Cross-Entropy (BCE) loss. This corresponds to modeling each pixel as a Bernoulli random variable and is appropriate when pixel intensities are interpreted probabilistically.

An alternative would be to model pixel intensities using a Beta distribution, which allows continuous values in $(0, 1)$ and provides greater flexibility. In this case, the decoder would output the parameters of the Beta distribution, and training would maximize the corresponding log-likelihood.

3.3 Multiple runs

Running the autoencoder multiple times results in qualitatively similar reconstructions but different latent space orientations. This variability is expected because the latent representation is not uniquely identifiable and can rotate or reflect without affecting reconstruction quality.

4 PCA Comparison

4.1 Qualitative comparison

Principal Component Analysis (PCA) provides a linear dimensionality reduction baseline. PCA reconstructions tend to be blurrier than those produced by the autoencoder, and the 2D latent scatter plot shows more overlap between digit classes.

The autoencoder, being nonlinear, learns a more structured latent representation and produces sharper reconstructions.

4.2 Determinism and reconstruction error

PCA is deterministic given fixed data and preprocessing, so repeated runs produce identical results. When varying the number of principal components or latent dimensions from 2 to 20, reconstruction error decreases for both PCA and the autoencoder. However, the autoencoder typically achieves lower reconstruction error with fewer latent dimensions.

There is no closed-form method to select the optimal latent dimensionality; common approaches include examining reconstruction error curves or using validation performance.

5 Variational Autoencoder

5.1 Model description

In the variational autoencoder (VAE), the encoder outputs both a mean vector μ and a log-variance vector $\log \sigma^2$ that define a Gaussian distribution in latent space. Latent variables are sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

The decoder maps samples from this latent distribution back to the data space.

5.2 Loss function

The VAE is trained by minimizing the negative Evidence Lower Bound (ELBO):

$$\mathcal{L} = \text{Reconstruction Loss} + \frac{1}{2} \sum \left(e^{\log \sigma^2} + \mu^2 - 1 - \log \sigma^2 \right),$$

where the first term measures reconstruction quality and the second term is the KL divergence between the approximate posterior and the standard normal prior.

5.3 Sampling and qualitative results

Unlike the standard autoencoder, the VAE learns a continuous and structured latent space that allows sampling. Images generated by sampling from a standard normal distribution are visually plausible but slightly blurrier than those produced by the autoencoder.

Repeated sampling produces different images due to the stochastic nature of the latent variables, while maintaining overall consistency in digit structure.